Prácticas Algoritmia

Irene Díaz Arias
UO306771

Contenido

Práctica 0 2
Práctica 1.1 7
Práctica 1.2 11
Práctica 2 13
Práctica 3 16
Práctica 4 21
Práctica 5 22
Práctica 6 23
Práctica 6 Extra 24
Práctica 7 25

Práctica 0

Algoritmo 1

Python :

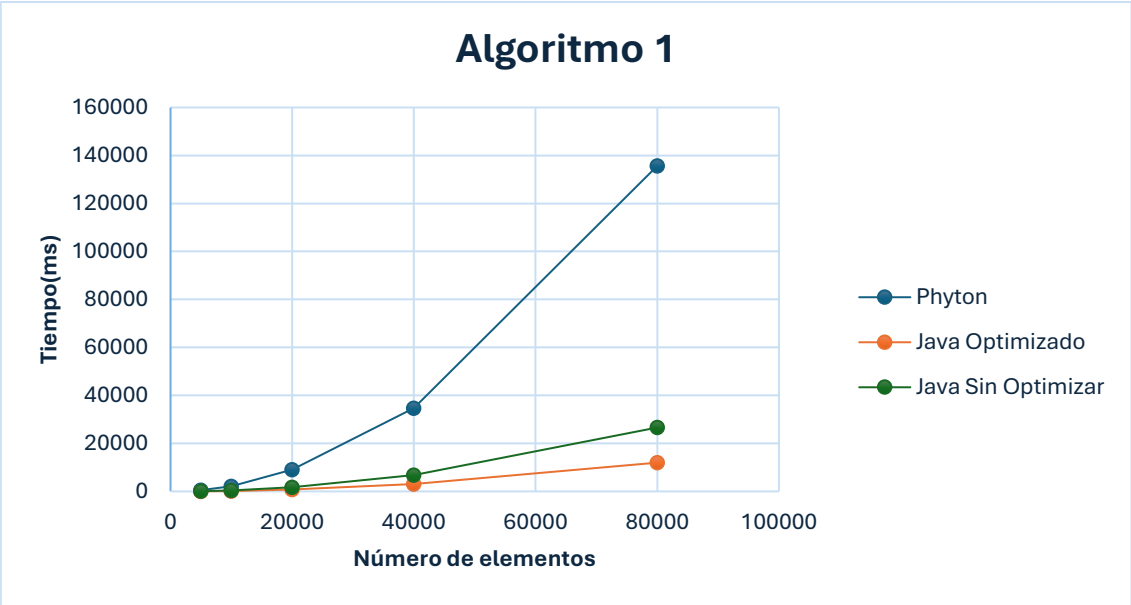
Número de elementos	Tiempo (milisegundos)
5000	356
10000	1511
20000	6124
40000	24712
80000	922884

Java sin optimización:

Número de elementos	Tiempo (milisegundos)
5000	100
10000	417
20000	1675
40000	6795
80000	26625

Java con optimización :

Número de elementos	Tiempo (milisegundos)
5000	42
10000	192
20000	767
40000	3068
80000	11988



Ordenador Utilizado :

Procesador	AMD Ryzen 5 1600X Six-Core Processor	3.60 GHz
RAM instalada	8,00 GB	

Resultados del algoritmo 1 :

Número de elementos	Tiempo (milisegundos)
5000	529
10000	2096
20000	9017
40000	34650
80000	135648

Algoritmo 2 :

PythonA2:

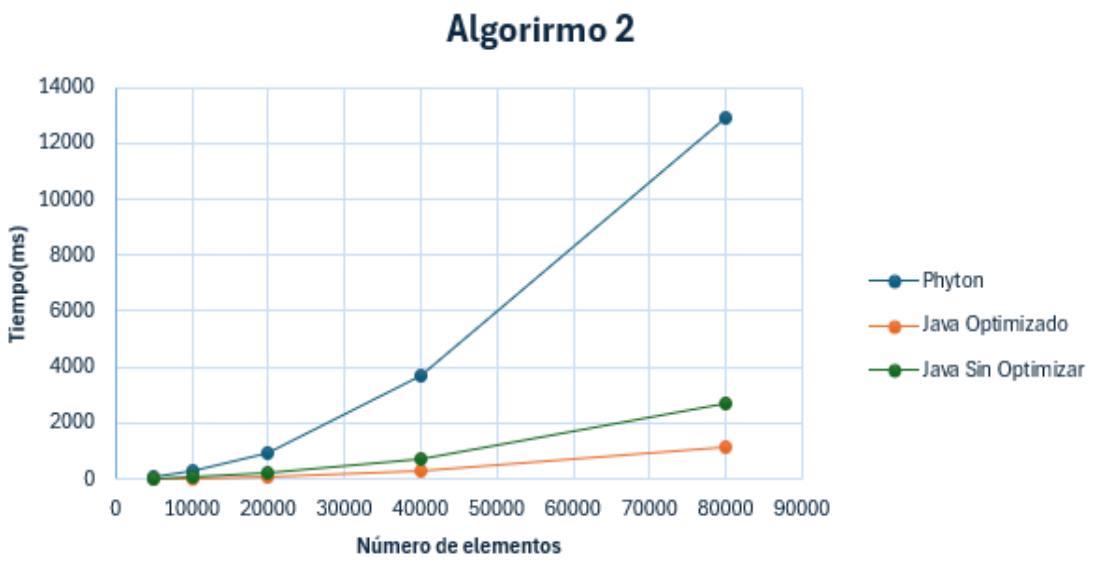
Número de elementos	Tiempo (milisegundos)
5000	104
10000	261
20000	918
40000	3663
80000	12937

JavaA2 sin optimización:

Número de elementos	Tiempo (milisegundos)
5000	17
10000	50
20000	184
40000	683
80000	2713

JavaA2 con optimización:

Número de elementos	Tiempo (milisegundos)
5000	7
10000	22
20000	83
40000	302
80000	1136



Algoritmo 3 :

PythonA3:

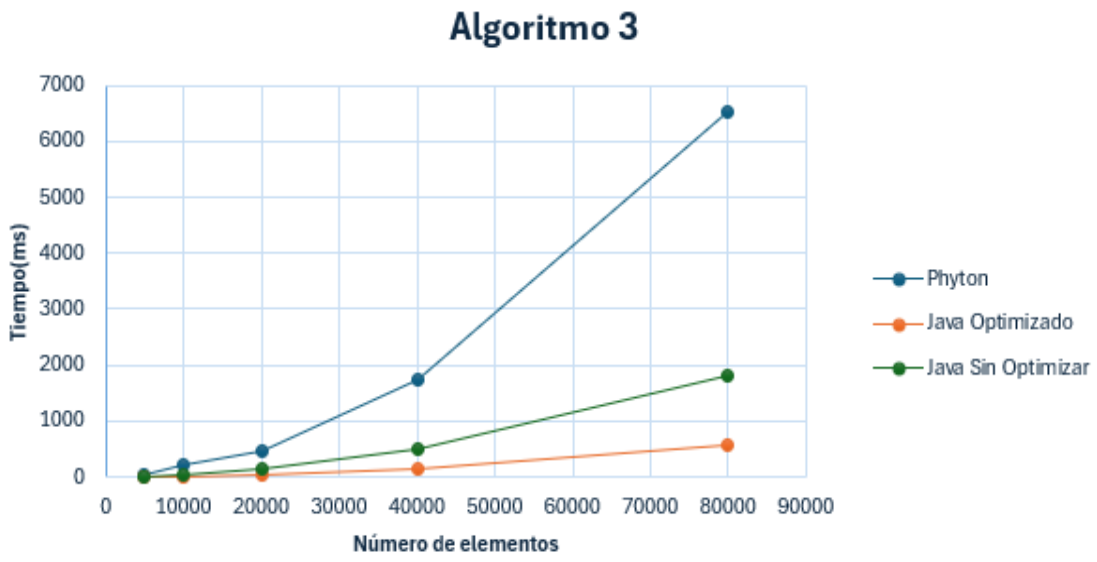
Número de elementos	Tiempo (milisegundos)
5000	34
10000	204
20000	467
40000	1729
80000	6519

JavaA3 sin optimización :

Número de elementos	Tiempo (milisegundos)
5000	8
10000	42
20000	136
40000	487
80000	1800

JavaA3 con optimización :

Número de elementos	Tiempo (milisegundos)
5000	8
10000	9
20000	50
40000	150
80000	584



Algoritmo 4

PythonA4:

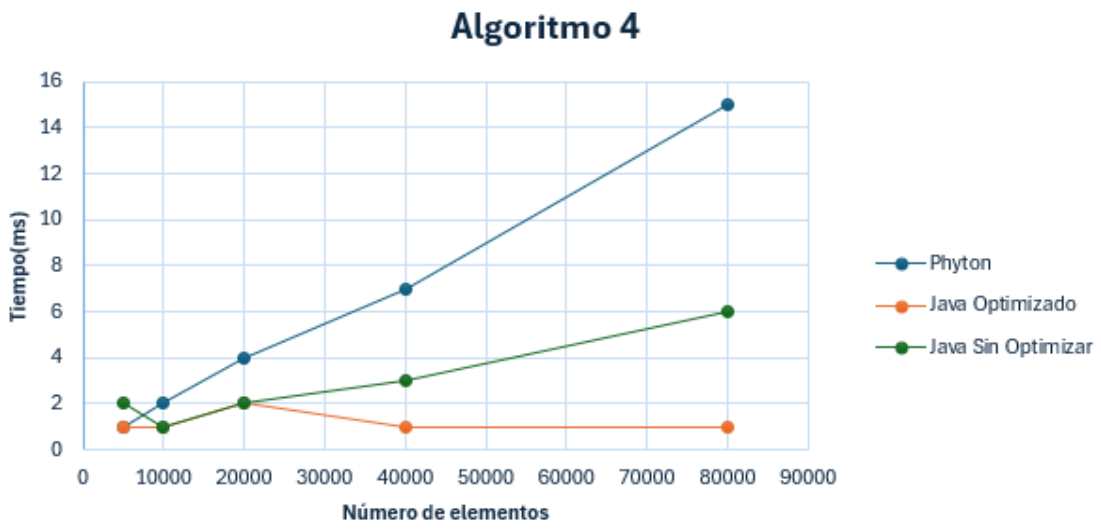
Número de elementos	Tiempo (milisegundos)
5000	1
10000	2
20000	4
40000	7
80000	15

JavaA4 Sin optimización:

Número de elementos	Tiempo (milisegundos)
5000	2
10000	1
20000	2
40000	3
80000	6

JavaA4 con optimización :

Número de elementos	Tiempo (milisegundos)
5000	1
10000	1
20000	2
40000	1
80000	1


Algoritmo 4
Tiempo(ms)
16
14
12
10
8
6
4
2
0
0
10000
20000
30000
40000
50000
60000
70000
80000
90000
Número de elementos
Phyton
Java Optimizado
Java Sin Optimizar

Práctica 1.1

Ejercicio 1

¿calcular cuántos años más podremos seguir utilizando esta forma de contar? Explica el razonamiento seguido para realizar el cálculo.

Resultado: 5,8 * 10^8 – 56

Ejercicio 2

Número de Elementos	Tiempo (milisegundos)
200 millones	983
50 millones	239
100 millones	490
20 millones	96
10 millones	49
1 millón	5
500 millones	2440

¿Por qué a veces el tiempo medido sale 0? ¿A partir de qué tamaño de problema (n) empezamos a obtener tiempos fiables?

El tiempo medido a veces sale 0 debido a procesos como el recolector de basura que interfieren en la medición. Empezamos a obtener tiempos fiables a partir de n=20 millones ya que el tiempo medido supera los 50 ms.

Ejercicio 3

Mediciones Vector 4

Repeticiones : 1

Número de Elementos	Tiempo (milisegundos)
80000	0
160000	1
320000	2
640000	3
1280000	6
2560000	12
5120000	24
10240000	55
20480000	98
40960000	195
81920000	393

Repeticiones : 10

Número de Elementos	Tiempo (milisegundos)
80000	10
160000	20
320000	30
640000	54
1280000	118
2560000	250
5120000	455
10240000	937
20480000	1859
40960000	3617
81920000	7446

Repeticiones : 100

Número de Elementos	Tiempo (milisegundos)
80000	80
160000	140
320000	318
640000	593
1280000	1161
2560000	2287
5120000	4606
10240000	9107
20480000	18318
40960000	36339
81920000	75379

Repeticiones : 1000

Número de Elementos	Tiempo (milisegundos)
20000	175
40000	367
80000	726
160000	1446
320000	2850
640000	5641
1280000	11428
2560000	22635
5120000	45729
10240000	92589
20480000	183012
40960000	360336

Repeticiones : 10000

Número de Elementos	Tiempo (milisegundos)
10000	9317
20000	18599
40000	37515
80000	85481
160000	146946
320000	292587
640000	591946

Repeticiones : 10 000 000

Número de Elementos	Tiempo (milisegundos)
10000	897550
20000	1794113
40000	3590823
80000	7158742
160000	14108528
320000	28176519

¿Qué pasa con el tiempo si el tamaño del problema se multiplica por 2?

El tiempo también aumentaría considerablemente pero su complejidad no cambiaría.

¿Qué pasa con el tiempo si el tamaño del problema se multiplica por otro k que no sea 2?

El tiempo del problema aumentaría proporcionalmente a la K que multipliquemos, es decir, cuanto mayor sea la k mayor serán los tiempos medidos.

Mediciones Vector 5 Con 100 Repeticiones

Número de Elementos	Tiempo (milisegundos)
80000	100
160000	201
320000	401
640000	807
1280000	1571
2560000	3174
5120000	6321
10240000	12499
20480000	25408
40960000	50746
81920000	101652

Mediciones Vector 6 Con 1 Repetición

Número de Elementos	Tiempo (milisegundos)
10000	792
20000	3168
40000	12645
80000	50676
160000	202377
320000	811440
640000	3136178
1280000	126321190

Mediciones Vector 7 Con 100 Repeticiones

Número de Elementos	Tiempo (milisegundos)
10000	20
20000	20
40000	50
80000	100
160000	200
320000	480
640000	824
1280000	1618
2560000	3317
5120000	6528
10240000	12983
20480000	25983
40960000	51974
81920000	102428

Práctica 1.2

Ejercicio 1

FdT : Fuera de Tiempo

N.º Elementos	Bucle 1	Bucle 2	Bucle 3	Bucle 4
100	0	1	1	1
200	0	0	4	6
400	0	2	15	42
800	0	15	63	325
1600	0	51	294	2558
3200	1	253	1112	206654
6400	0	1058	4809	164670
12800	2	4696	20086	FdT
25600	4	21169	83493	FdT
51200	7	82969	FdT	FdT
102400	15	333846	FdT	FdT
204800	33	FdT	FdT	FdT
409600	61	FdT	FdT	FdT
819200	140	FdT	FdT	FdT

Ejercicio 2

N.º Elementos	Bucle 5	Bucle 6	Bucle 7
100	1	FdT	FdT
200	4	FdT	FdT
400	23	FdT	FdT
800	108	FdT	FdT
1600	516	FdT	FdT
3200	2457	FdT	FdT
6400	11560	FdT	FdT
12800	52960	FdT	FdT

Ejercicio 3

Algoritmos Diferente Complejidad

n	t1	t2	t1/t2
100	0	1	0
200	0	0	0
400	0	2	0
800	0	15	0
1600	1	51	0,01960784
3200	1	253	0,00395257
6400	2	1058	0,00189036
12800	2	4696	0,00042589
25600	4	21169	0,00018896
51200	7	82969	8,4369E-05
102400	15	333846	4,4931E-05

El cociente de ambos tiempos cuadra con lo esperado ya que bucle 1 tiene menor complejidad que bucle 2 por lo que su cociente tenderá a 0.

Algoritmos Misma Complejidad

n	t3	t2	t3/t2
100	1	1	1
200	4	1	4
400	15	2	7,5
800	63	15	4,2
1600	294	51	5,76470588
3200	1112	253	4,39525692
6400	4809	1058	4,54536862
12800	20086	4696	4,27725724
25600	83493	21169	3,9441164

El cociente de ambos tiempos cuadra con los esperado ya que tiende hacia un valor constante, que relaciona ambos algoritmos.

Mismo Algoritmo En Entornos De Desarrollo Diferentes

n	tPython	tJava	tPython/tJava
200	21	6	3,5
400	159	42	3,785714286
800	1398	325	4,301538462
1600	12107	2558	4,732994527
3200	108559	20654	5,256076305

Práctica 2

Algoritmo Burbuja

El algoritmo recorre la lista con dos bucles y va comparando elementos consecutivos y si el primero es mayor que el segundo los intercambia. Su eficiencia puede cambiar según el estado inicial de la lista :

- **Vector ordenado :** nunca va a realizar el intercambio de valores. Es el caso más rápido
- **Vector orden inverso :** siempre se va a realizar el intercambio de valores. Es el caso más lento
- **Vector orden aleatorio** : será un tiempo comprendido entre los anteriores.

n	tOrdenado	tInverso	tAleatorio
10000	438	1104	865
20000	1676	4418	3487
40000	6845	17535	13971
80000	27183	69937	56495
160000	109358	279215	224831

Algoritmo Selección

El algoritmo siempre va a realizar el mismo número de comparaciones independientemente del estado original del vector por lo que los tiempos no van a tener variaciones muy grandes entre ellas.

n	tOrdenado	tInverso	tAleatorio
10000	414	351	415
20000	1584	1384	1585
40000	6387	5661	6353
80000	25661	22219	25350
160000	102341	88954	100600

Algoritmo Inserción

Este algoritmo itera sobre cada elemento y lo coloca en la posición correspondiente según su valor. Su comportamiento cambia según el estado inicial del vector:

- **Ordenado :** apenas realiza comparaciones por lo que es muy rápido.
- **Orden inverso**: tiene que mover todos los elementos al sitio correspondiente por lo que el tiempo más lento.
- **Orden Aleatorio** : El tiempo no es tan lento como en el orden inverso pero tarda un tiempo considerable.

n	TOrdenado	tInverso	tAleatorio
10000	1	693	342
20000	3	2788	1363
40000	6	11015	5475
80000	10	43837	21838
160000	34	173704	86910

Algoritmo Rápido (repeticiones=10)

n	TOrdenado	tInverso	tAleatorio
10000	24	29	36
20000	50	57	78
40000	107	117	165
80000	224	251	365
160000	480	526	761

Algoritmo Rápido vs. Inserción Tiempos Bajos

n	tRapido	tInsercion	tRapido/tInsercion
10	113	58	1,9483
15	175	111	1,5766
20	275	168	1,6369
25	356	277	1,2852
30	445	369	1,2060
35	531	491	1,0815
40	640	659	0,9712
45	744	777	0,9575
50	878	947	0,9271
55	939	1148	0,8179
60	1052	1396	0,7536
65	1148	1584	0,7247
70	1318	1855	0,7105
75	1341	2128	0,6302
80	1457	2366	0,6158
85	1602	2588	0,6190
90	1739	2968	0,5859
95	1805	3278	0,5506
100	1946	3657	0,5321

A partir de n=40 el algoritmo rápido comienza a tardar menos tiempo que el de inserción.

Proyecte, a partir de las complejidades y los datos de las tablas anteriores, ¿cuántos días tardaría cada uno de los cuatro métodos en ordenar un vector de n= 811920.000, en el caso inicialmente en orden aleatorio?

Método	Complejidad	Tiempo estimado
Burbuja	O(n^2)	6755,40
Selección	O(n^2)	3031,23
Inserción	O(n^2)	2611,28
Rápido	O(n*log n)	43,64

Práctica 3

¿para qué valor de n dejan de dar tiempo las clases Sustracción1 y Sustracción2? ¿por qué sucede eso?

Sustración1 deja de dar tiempo para n=8000 y Sustración2 para n=8000. Se produce un StackOverflowError porque se acaba la memoria dentro de la pila de la máquina virtual de Java.

¿cuántos años tardaría en finalizar la ejecución Sustracción3 para n=80? Razonar la respuesta.

Para n=80 el tiempo que tardaría es T(n)*2^(80-n). Teniendo en cuenta que para n=22 T= 1,66 segundos entonces para n=80 el tiempo en años sería de 15171943950.

Implementar una clase Sustracción4.java con una complejidad O(n3) (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=100, 200, 400, 800, ... (hasta FdT).

n	Tiempo(ms)
100	3
200	21
400	145
800	1113
1600	8877
3200	72879

```java
public static void rec4 (int n) // O(n^2)
{
    if (n<=0)
        cont++;
    else
    {
        for (int i=0;i<n;i++) cont++; // O(n)
        for (int i=0;i<n;i++){
            for (int j=0;j<n;j++){
                cont++;
            }
        }

        rec4 (n-1);
    }
}
```

Implementar una clase Sustracción5.java con una complejidad O(3^n/2) (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=30, 32, 34, 36, ... (hasta FdT).

Para tener esa complejidad: a=3 y b=2

n	Tiempo(ms)
30	397
32	1198
34	3584
36	11001
38	32952
40	97522

```
public static void rec5 (int n) // O(n^2)
{
    if (n<=0)
        cont++;
    else
    {
        cont++;
        rec5(n-2);
        rec5(n-2);
        rec5(n-2);
```

¿cuántos años tardaría en finalizar la ejecución Sustracción5 para n=80? Razonar la respuesta.

Cada vez que n aumenta en 2 el tiempo de ejecución se triplica. De n=40 a n=80 hay una diferencia de 40 unidades, como los saltos son de dos en dos habría 20 saltos de crecimiento. Por lo que para n=80 sería igual a T(40)*3^20 = 10782,54 años.

Implementar una clase Division4.java con una complejidad O(n2) (con a<b^k) y la pertinente toma de tiempos. Después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=1000, 2000, 4000, 8000, ... (hasta FdT).

n	Tiempo(ms)
1000	10
2000	38
4000	141
8000	563
16000	2232
32000	9388
64000	37305

```
public static void rec4 (int n) //O(n^2)
{
    // a=1
    // b=2
    // k=2
    // a<b^k
    // Complejidad : O(n^2)
    if (n<=0)
        cont++;
    else
    {
        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
                cont++;
            }
        }
        rec1 (n/2);
```

Implementar una clase Division5.java con una complejidad O(n2) (con a>b^k) y la pertinente toma de tiempos. Después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=1000, 2000, 4000, 8000, ... (hasta FdT)

n	Tiempo(ms)
1000	32
2000	127
4000	501
8000	2007
16000	8194
32000	33021
64000	131102

```
public static void rec5 (int n)
    {
        // a=4
        // b=2
        // k=1
        // a>b^k
        // Complejidad : O(n^log2(4))=O(n^2)
        if (n<=0)
            cont++;
        else
        {
            for(int i=0;i<n;i++){
                cont++;
            }
            rec5 (n/2);
            rec5 (n/2);
            rec5 (n/2);
            rec5 (n/2);
```

Mediciones de tiempos Vector Suma (n=100)

Complejidad Método 1: iterativo O(n)

Complejidad Método 2: recursivo Sustracción (a=1;b=1;k=0)=>O(n)

Complejidad Método 3: recursivo División (a=2;b=2;k=0)=>O(n)

Método	Tiempo(ms)
1	523
2	1624
3	2922

Mediciones de tiempos Fibonacci

Complejidad Método 1: iterativo O(n)

Complejidad Método 2: iterativa O(n)

Complejidad Método 3: recursiva Sustracción (a=1;b=1;k=0)=>O(n)

Complejidad Método 4: recursiva Sustracción (a=2;b=1 o b=2;k=0)=>O(1.6**n)

Método	Tiempo(ms)
1	236
2	492
3	667
4	4243

Mediciones Tiempos Puntos Trivial

n	Tiempos
1024	24
2048	20
4096	31
8192	135
16384	567
32768	3032
65536	9071
131072	46560
262144	190450

Mediciones Tiempos Puntos Divide y Vencerás

n	Tiempos
1024	0
2048	0
4096	9
8192	0
16384	16
32768	25
65536	42
131072	83
262144	175

Práctica 4

Medición de Tiempos Colorear Grafos JAVA

N	Tiempo
8	4
16	0
32	1
64	4
128	3
256	4
512	7
1024	5
2048	5
4096	16
8192	16
16384	39
32768	42
65536	91

Práctica 5

Proyecto Ferry

1. **Definición de Estado:**

Cada columna representa el número de metros que están siendo ocupados en babor y las filas indican que coches se quiere meter dentro de ese lado del barco. Si una casilla (fila,colum) es verdadera significa que para el número de metros de la columna cabe el coche correspondiente a la fila dónde se encuentra.

2. **Relación de recurrencia:**

Primero definimos el caso base, sabemos que la primera casilla dp[0][0]= true y el resto de la fila es false ya que para cero coches no se van a ocupar más metros.
Para cada coche i y longitud L tenemos dos casos :

1. Hay sitio en babor : l + vehiculo(i-1) <= boatLength
   Sabemos que el coche cabe para la longitud calculada por lo que d[i][l + vehiculo(i-1)]=true
2. Hay sitio en estribor: sumatorio[i] - l <= boatLength.
   Sabemos que el coche para longitud entra en estribor por lo que dp[i][l] = true;

La ejecución para cuando no entra ningún coche más para la longitud establecida inicialmente.

Práctica 6

TIEMPOS MEDIDOS ALMACENAJE CONTENEDORES

Archivo	Tiempos(ms)	Num Llamdas
test00.txt	0	136
test01.txt	0	536
test02.txt	6	19544
test03.txt	125	1036339
test04.txt	25937	222281543

¿Qué complejidad asigna al algoritmo? ¿Por qué?

Para calcular la complejidad de un algoritmo de backtracking usamos la fórmula $O(B^p)$ dónde:

- B= Número máximo de estados que se exploran en cada decisión
- P=Altura máxima del árbol (Número de transiciones de estado antes de llegar a una solución válida)

En este caso B es el número de posibles formas que existen de colocar un nuevo elemento dentro de los contenedores, es decir, dos (en uno nuevo o en uno viejo). Por otro lado, el valor de p es igual al número de objetos que se quieran meter en los contenedores, es decir, n. De esta forma, la complejidad nos quedaría: $O(2^n)$

Práctica 6 Extra

Mediciones de Tiempos NReinas

Sin optimización

Tamaño Tablero	Tiempo(ms)
4	0
6	0
8	2
10	10
12	368
14	14336

Con optimización

Tamaño Tablero	Tiempo(ms)
4	2
6	0
8	0
10	7
12	129
14	3380

Práctica 7

Mediciones Tiempos Almacenaje Contenedores Con Poda

Archivo	Tiempo	NumLlamadas
0	0	8
1	0	10
2	0	16
3	0	13
4	0	21
5	0	27
6	0	15
7	0	16
8	0	42
9	0	67